

Cache-Aware Memory Management for On-Device Personalized Assistants

Anonymous ACL submission

Abstract

온디바이스 개인화 비서는 지속적인 대화에서 변화하는 사용자 선호를 정확하게 관리하면서 제한된 연산 자원 안에서 메모리를 갱신해야 한다. 본 연구는 턴별 갱신 판단과 메모리 상태 정답, 대화 중간 및 종료 시점의 선호 기반 도구 호출 평가를 제공하는 벤치마크 CAMPBench를 구축하고, 캐시 재사용을 고려한 메모리 관리 방식인 CAMP를 제안한다. CAMP는 기존 메모리를 안정적인 prefix로 유지하고 변경 연산을 순차적으로 누적한 뒤 주기적으로 병합하며, 병합 전에도 변경사항을 즉시 활용할 수 있도록 한다. 온디바이스 환경을 대상으로 350M-3B 규모의 여섯 언어 모델의 메모리 관리 성능을 평가하고, 모바일 기기에서 처리 비용을 직접 측정하였다. 기본 병합 주기($k=5$)에서 CAMP는 즉시 부분 수정 방식인 Patch 대비 갱신 생성과 직후 입력 처리를 포함한 UPDATE-cycle 지연시간을 평균 24.7% 줄였으며, 종합 평가 점수는 평균 2.28점 낮았다. 캐시 활성화·비활성화 비교는 이러한 효율성 이점이 prefix 재사용에서 발생함을 뒷받침하였다. 또한 병합 주기 분석을 통해 정확도와 지연시간 사이의 절충을 조절할 수 있음을 확인하였다. 이러한 결과는 온디바이스 메모리 관리에서 갱신 표현과 병합 시점을 정확도 및 추론 비용과 함께 설계할 필요가 있음을 보여준다.

1 Introduction

차량 AI 비서와 같은 개인화 서비스는 사용자와의 지속적인 상호작용에서 선호와 설정을 기억하고, 이를 이후 요청의 해석과 도구 호출에 반영해야 한다. 이러한 서비스에서는 사용자 정보가 외부로 전송되는 범위와 지속적인 추론에 필요한 운영 비용이 중요한 고려사항이다. 온디바이스 소형 언어모델은 개인화 메모리를 기기 내부에서 관리함으로써 외부 전송과 클라우드 추론 의존도를 줄일 수 있다 (Apple, 2024). 따라서 제한된 기기 자원 안에서 사용자 정보를 정확하게 유지하고 활용하는 메모리 관리 방법이 필요하다.

장기 대화에서 사용자 정보를 추출하고 갱신하는 연구는 지속적으로 이루어져 왔다(Packer

et al., 2023; Chhikara et al., 2025). 그러나 온디바이스 환경에서는 소형 모델의 제한된 정보 처리 능력뿐 아니라 메모리 용량과 연산 성능도 고려해야 한다(Alizadeh et al., 2024). 긴 대화 이력에서 캐시로 재사용되지 않는 입력을 반복 처리하면 prefill 비용이 발생하며, 메모리 전체를 재작성하면 변경분만 생성하는 경우보다 decode해야 할 토큰 수가 증가한다 (Agrawal et al., 2024; Zheng et al., 2024). 또한 기존 메모리를 수정하는 위치와 방식에 따라 prefix KV-cache의 재사용 범위가 달라질 수 있다. 따라서 온디바이스 메모리 관리에서는 어떤 정보를 보존하는지와 함께, 변경사항을 어떻게 표현하고 언제 기존 상태에 병합하는지가 정확도와 지연시간을 결정하는 중요한 설계 요소가 된다.

이러한 메모리 갱신 구조의 정확도-지연시간 절충을 평가하려면, 처리 비용과 함께 대화 중 각 시점의 메모리 상태가 올바르게 갱신·유지되는지를 확인해야 한다. LongMemEval과 VehicleMemBench 등은 정보 갱신과 기억 활용 능력을 평가하지만(Wu et al., 2025; Chen et al., 2026), 답변 시점의 정확도만으로는 정보가 언제 누락되거나 잘못 갱신되었는지 진단하기 어렵다. 사용자는 대화 도중 언제든지 자신의 선호에 기반한 동작을 요청할 수 있으므로, 최종 메모리의 품질과 함께 중간 시점에서 필요한 정보가 올바르게 유지되는지도 평가해야 한다. 특히 갱신이 필요하지 않은 대화에서는 기존 상태를 보존하고, 선호가 변경된 시점에는 이를 적절하게 반영하는 능력이 중요하다.

이를 위해 본 연구는 턴별 메모리 관리 평가에 집중한 CAMPBench 벤치마크를 소개한다 (Chen et al., 2026). CAMPBench는 각 턴의 메모리 갱신 여부와 메모리 상태에 대한 supervision을 제공하며, 대화 중간과 종료 시점의 선호 기반 도구 호출 문제를 통해 기억의 실제 활용 성능을 평가한다.

본 연구는 전체 메모리를 재작성하는 Recursive Summary와 변경 부분을 즉시 반영하는 Patch를 비교 기준으로 삼고, 온디바이스 소형 언어모델을 위한 cache-aware 메모리 구성 방식인 CAMP (Cache-Aware Memory Patch)를 제안한

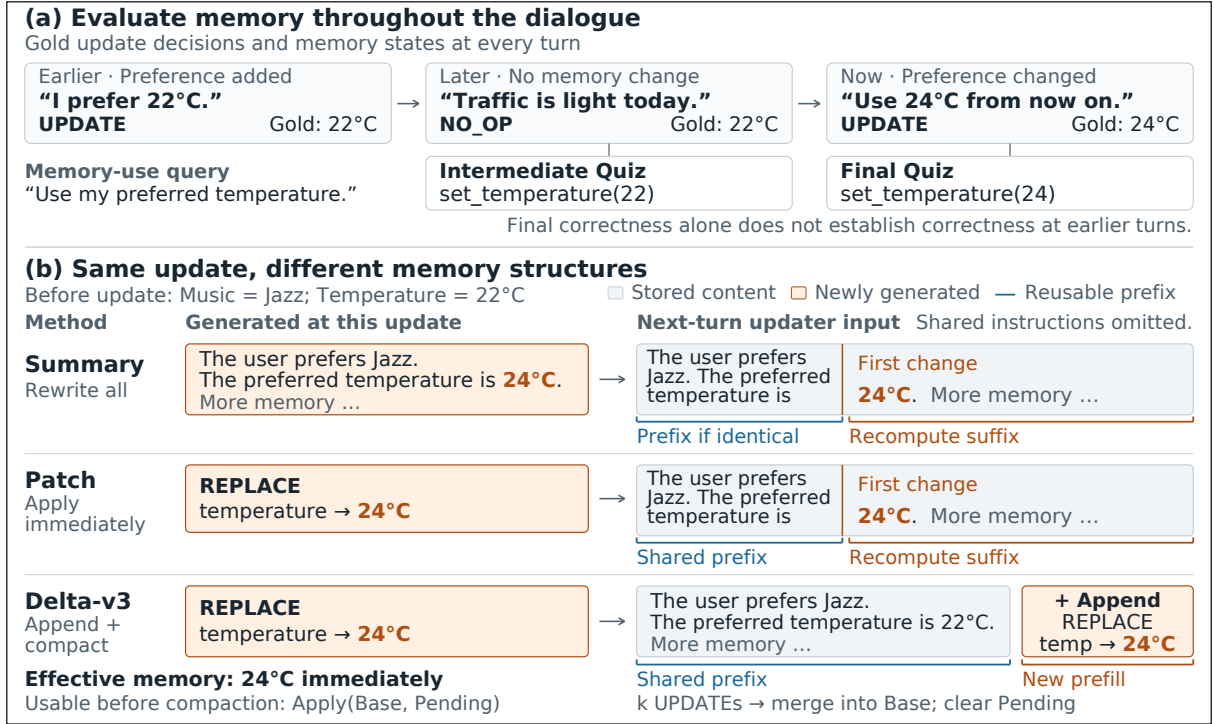


Figure 1: (a) Turn-level memory evaluation. (b) Memory update structures and prefix KV-cache reuse.

다. Recursive Summary는 매 갱신 시 전체 메모리를 재작성하므로 decode 비용이 높다. Patch는 변경 연산만 생성하여 출력 비용을 줄이지만, 수정된 메모리를 다음 입력에 반영하는 과정에서 prefix KV-cache의 재사용 범위가 제한될 수 있다. CAMP는 변경 연산의 누적과 주기적 병합을 통해 prefix KV-cache 재사용을 높이도록 설계하였다. 변경사항은 병합 전에도 즉시 활용되며, 병합 주기에 따른 정확도-지연시간 절충을 분석한다. Figure 1은 턴별 메모리 평가 구성(a)과 갱신 방식에 따른 prefix KV-cache 재사용 범위의 차이(b)를 보여준다.

실험에서는 비학습 기준선과 Mem0 계열 방법을 포함하여 메모리 관리 성능을 비교하고, 주요 세 방법의 정확도와 온디바이스 처리 비용을 분석한다 (Chhikara et al., 2025). 메모리 갱신 판단, 최종 상태의 정확성, 선호 기반 도구 호출 성능을 함께 평가하며, 처리 비용에는 주기적 병합에 따른 비용도 포함한다. 이를 바탕으로 모델 크기, 업데이트 부하, 병합 주기에 따른 정확도-지연시간의 Pareto 관계를 분석하여, 제한된 기기 자원에서 적절한 메모리 관리 방식을 선택할 근거를 제시한다.

본 논문의 기여는 다음과 같다.

1. 턴별 메모리 관리 벤치마크: 차량 사용자 선호에 대한 메모리 업데이트 판단과 메모리 상태 supervision, 중간·최종 시점의 도구 호출 평가를 제공하는 CAMPBench를 소개한다.

2. 캐시 재사용을 고려한 메모리 구성: 온디바이스 소형 언어모델을 위해 안정적인 base prefix와 변경 연산의 누적·주기적 병합을 결합한 CAMP를 제안한다.

3. 온디바이스 효율성과 정확도-지연시간 분석: 실제 온디바이스 환경에서 KV-cache 재사용과 입력 재계산량을 측정하여 CAMP의 속도 향상 기전을 검증한다. 병합 비용을 포함하여 모델 크기, 업데이트 부하, 병합 주기에 따른 정확도-지연시간의 Pareto 관계를 분석한다.

2 Related Work

2.1 Long-Term Memory Management

장기 대화의 개인화를 위한 메모리 연구는 대화에서 필요한 정보를 추출하고, 이를 지속적으로 조직·갱신하여 이후 요청에 활용하는 방향으로 발전해 왔다. MemGPT는 계층적 메모리와 명시적인 데이터 이동을 통해 제한된 컨텍스트 안에서 장기 상호작용을 지원한다 (Packer et al., 2023). Mem0는 대화의 주요 정보를 추출·통합·검색하는 메모리 관리 구조를 제시한다 (Chhikara et al., 2025).

메모리 간 관계와 시간에 따른 변화를 관리하는 연구도 이루어지고 있다. A-MEM은 구조화된 메모리 노트 사이의 연결을 생성하고, 새로운 정보에 따라 기존 메모리의 표현과 속성을 갱신한다 (Xu et al., 2025). Zep은 시간 정보를 반영한 지식 그래프를 통해 변화하는 사실과 역사적 관

계를 관리한다 (Rasmussen et al., 2025). 이들은 메모리의 조직과 갱신이 장기적인 정보 활용에 중요한 설계 요소임을 보여준다.

본 연구는 소형 모델의 턴별 메모리 관리에 초점을 맞추어, 전체 재작성·즉시 부분 수정·변경 분 누적 및 병합이라는 갱신 구조의 차이가 기억의 정확성과 처리 비용에 미치는 영향을 비교한다.

2.2 Memory Evaluation and Benchmarks

장기 메모리 벤치마크는 축적된 대화 정보가 이후 과제 수행에 얼마나 효과적으로 활용되는지를 평가한다. LoCoMo는 장기 다중 세션 대화에서 질문 응답, 사건 요약 및 멀티모달 대화 생성을 통해 기억 능력을 평가한다 (Maharana et al., 2024). LongMemEval은 정보 추출, 다중 세션 추론, 시간 추론, 지식 갱신, 답변 유보의 다섯 능력을 평가하며, 변화하는 사용자 정보를 기억하고 활용하는 문제를 포함한다 (Wu et al., 2025). VehicleMemBench는 차량 환경의 다중 사용자 선호와 시간에 따른 변화를 다루며, 도구 실행 이후의 환경 상태를 목표 상태와 비교하여 기억에 기반한 동작의 정확성을 평가한다 (Chen et al., 2026).

이러한 과제 수준의 평가는 기억의 활용 성능을 측정하지만, 답변이나 실행 결과만으로는 대화 중 어느 시점에 정보가 누락되거나 잘못 수정되었는지를 구분하기 어렵다. 본 연구의 CAMP-Bench는 턴별 갱신 여부와 메모리 상태 정답을 제공하고, 선택된 중간 시점과 종료 시점의 도구 호출을 평가하여 메모리 관리 과정과 활용 성능을 연결한다.

2.3 Efficient Memory Management and KV-Cache Reuse

메모리 시스템의 효율성은 정보를 압축하는 정도뿐 아니라, 추출·통합·검색이 수행되는 방식과 시점에도 영향을 받는다. Mem0는 전체 대화 입력 방식과 비교하여 지연시간과 토큰 비용을 줄이는 메모리 관리 구조를 제시한다. LightMem은 정보 필터링과 주제별 통합을 수행하고, 장기 메모리의 통합을 온라인 추론과 분리하는 sleep-time update를 도입한다 (Fang et al., 2026). 이처럼 정보 처리량을 줄이고 통합 시점을 조정하는 접근은 메모리 관리의 정확도와 비용을 함께 고려한다.

추론 시스템에서는 반복 입력 간의 KV-cache 재사용을 통해 중복 연산을 줄이는 접근이 연구되어 왔다. SGLang의 RadixAttention은 이러한 캐시 재사용을 지원하는 대표적인 기법이다 (Zheng et al., 2024). Prompt Cache는 명시적으로 정의된 prompt 모듈의 attention state를 재사용하여 입력

처리 비용을 줄인다 (Gim et al., 2024). 이 관점에서 메모리의 갱신 위치와 입력 내 배치는 후속 추론에서 재사용할 수 있는 prefix의 범위와 연결되는 설계 요소로 볼 수 있다.

본 연구는 메모리 통합 시점과 캐시 재사용에 관한 이러한 연구를 바탕으로, 메모리 갱신의 표현과 병합 시점이 후속 입력의 prefix 재사용 범위에 미치는 영향을 다룬다. 또한 병합 비용을 포함한 정확도-지연시간 절충을 온디바이스 환경에서 평가한다.

3 Turn-Level Memory Evaluation

3.1 과제 정의

대화는 시간순으로 배열된 발화 ($x_1, \dots, x_T$)로 구성되며, 개별 화자의 발화 하나를 한 턴으로 정의한다. 메모리에 저장되는 각 선호는 사용자, 차량 속성, 적용 조건 및 값으로 표현한다. 턴 t 에서 메모리 모델은 이전 메모리 M_{t-1} 과 현재 발화 x_t 를 입력받아, 메모리 갱신 여부 $d_t \in \{\text{UPDATE}, \text{NO_OP}\}$ 를 판단한다. UPDATE인 경우에는 선호의 추가·변경·철회를 반영하여 새로운 메모리 상태 M_t 를 구성하고, NO_OP인 경우에는 기존 상태를 유지한다. ($M_t = M_{t-1}$).

각 평가 질의 (q)는 특정 대화 시점 (c_q)와 연결된다. 해당 시점의 메모리 (M_{c_q})와 질의를 바탕으로 하나 이상의 차량 도구 호출을 생성하는 것이 기억 활용 과제이다. Turn Quiz는 대화 중간 시점의 메모리를, Final Quiz는 전체 대화 종료 후의 메모리를 평가한다.

3.2 평가 설계

일반적인 전체 대화 이후의 최종 질의를 통해 장기 사용자 선호의 복원과 차량 도구 실행을 평가하는 방식을 넘어, CAMPBench는 기존의 다중 사용자 시나리오, 시간순 사건 구조, 공식 도구 스키마 및 차량 시뮬레이터를 유지하면서 평가 범위를 메모리 형성 과정과 중간 시점의 기억 활용으로 확장한다.

첫째, 모든 대화 턴에 UPDATE/NO_OP 라벨과 갱신 후 정답 메모리 상태를 제공한다. 이를 통해 갱신이 필요한 시점의 판단과 갱신 결과의 정확성을 구분하여 평가한다. 갱신이 불필요한 턴에서는 기존 상태를 보존하는지, 선호가 추가·변경·철회된 턴에서는 해당 변화를 올바르게 반영하는지 확인한다.

둘째, 최종 시점뿐 아니라 대화 중간에도 Quiz를 배치하여 유지된 기억이 실제 도구 호출에 활용되는지를 평가한다. Turn Quiz는 선호 갱신 직후의 immediate 질의, 시간이 경과한 후의 delayed 질의, 여러 선호를 함께 적용하는 composite 질의로 구성한다. 각 시나리오에는 30개의 Turn Quiz

와 10개의 Final Quiz를 배치한다. 이를 통해 메모리 관리 과정의 정확성과 중간·최종 시점의 활용 성능을 함께 평가한다.

3.3 데이터 구축

본 연구의 데이터셋은 기존 VehicleMemBench의 생성 방식을 참고하여 새롭게 구성하였다 (Chen et al., 2026). 각 시나리오는 세 명의 차량 이용자와 차량 선호에 관련된 사건 및 이와 무관한 배경 사건을 포함하며, 이를 시간순으로 배열하여 장기 대화를 생성하였다. 차량 설정은 공식 도구 스키마에서 지원하는 기능과 값으로 제한하였다.

대화 생성에 앞서 선호의 추가·변경·철회에 따른 정답 상태 전이를 구조화하였다. 이후 구조화된 사건을 자연어 대화로 변환하고, 각 상태 변화의 근거를 대화에 정렬하였다. 턴별 정답 생성에는 Hybrid 방식을 사용하였다. 하나의 사건에 해당하는 대화는 일괄 생성하되, 다음 사건으로 넘어가기 전에 내부 발화를 시간순으로 처리하여 정답 메모리 상태를 확정하였다. 모델은 선호의 추가·변경·철회가 확정되는 근거 턴을 식별하고, 규칙 기반 변환기는 해당 구조화된 사건을 ADD, REPLACE, DELETE 연산으로 변환하였다. 이 연산을 순차 적용하여 모든 턴의 갱신 여부와 갱신 후 정답 메모리 상태를 생성하였다.

근거 정렬 모델은 사건 전체의 대화를 입력받지만, 각 UPDATE가 지정된 근거 턴까지의 대화만으로 정당화되는지는 별도로 검사하였다. 또한 각 UPDATE의 정답 갱신이 직전 Gold 메모리와 현재 발화만으로 판단 가능한지 확인하였다. 근거 문장이 지정된 턴에 존재하지 않거나 해당 시점까지의 대화만으로 상태 변화를 확정할 수 없는 경우에는 근거 정렬 또는 대화 생성을 다시 수행하였다. 이를 통해 일부 턴을 제거하는 대신, 모든 턴을 포함하면서 각 시점의 근거에 부합하는 정답 메모리 궤적을 구성하였다.

데이터의 정합성을 확인하기 위해 메모리 연산 재실행 결과와 정답 상태의 일치, NO_OP 전후의 상태 불변성, 턴 간 메모리 상태의 연결을 자동으로 검사하였다. 또한 각 Quiz 시점에 정답을 뒷받침하는 근거가 존재하는지와 정답 도구 호출의 실행 가능성을 검사하였다.

전체 데이터는 100개 시나리오와 276,938개의 개별 발화 턴으로 구성된다. 이 가운데 1,308개는 UPDATE 턴이며, 총 1,398개의 메모리 연산을 포함한다. 전체 Quiz는 Turn Quiz 3,000개와 Final Quiz 1,000개로 구성된다.

데이터는 시나리오 단위로 분할하였으며, 분할 간 persona와 event chain이 중복되지 않도록 구성하였다. 구축 방식의 선택을 검증하기 위해, 동일한 persona와 구조화된 사건을 고정한 20개 시나리오에서 세 가지 생성 방식을 비교하였다.

Hybrid의 갱신 라벨 감사 통과율은 95.3%였으며, 시나리오당 생성 비용은 \$2.55로 Post-hoc(\$2.58)과 유사하였다. 이 결과를 바탕으로 CAMPBench 구축에 Hybrid를 채택하였으며, 전체 비교 결과와 세부 감사 절차 및 오류 분석은 부록에 제시한다.

3.4 Human 데이터 구축

생성한 데이터에서 학습한 모델의 외부 일반화 성능을 평가하기 위해, 40개 시나리오로 구성된 human-validated 평가 세트를 구축하였다. 외부 참조 자료로는 참가자들이 이상적인 음성 비서와의 상호작용을 상상하여 작성한 Envisioned Voice Assistant Dialogues를 활용하였다. (Volk et al., 2021). 또한 차량을 포함한 여러 도메인의 도구 호출 과제를 제공하는 Audio2Tool을 활용하였다 (Pahwa et al., 2026). 전자는 자연스러운 대화 표현을, 후자는 차량 기능, 오류 교정 및 다중 턴 상호작용을 구성하는 데 참고하였다.

외부 대화는 차량 메모리 과제의 형식에 맞게 정규화한 후, 독립적인 인간 작성자가 차량 상황과 지속적인 사용자 선호가 자연스럽게 드러나도록 재작성하였다. 별도의 annotator가 각 턴의 UPDATE/NO_OP, memory owner, 활성 메모리 상태 및 Gold Tool call을 검수하였으며, 불일치는 제3의 검수자가 조정하였다.

평가 세트는 두 난이도로 구성하였다. Human Standard Test(HST)는 명시적인 단일 사용자 선호를 평가하며, 20개 시나리오, 600개 대화 턴 및 320개 Quiz를 포함한다. Human Challenge Test(HCT)는 사용자 간 선호 충돌, 공지시어 해소, 선호 변경 및 복합 요청을 포함하며, 20개 시나리오, 1,174개 대화 턴 및 574개 Quiz로 구성된다.

최종 데이터는 메모리 상태 전이, Quiz 시점의 정답 근거, Tool schema를 검증하였다. HST와 HCT는 학습과 checkpoint 선택에 사용하지 않고, 모든 모델에 동일하게 적용되는 zero-shot 외부 평가 세트로 사용하였다.

4 Memory Update Methods

각 방법은 턴별로 UPDATE 또는 NO_OP를 판단하며, NO_OP로 판단한 경우 기존 메모리 상태를 유지한다. Quiz 평가에서는 각 방법의 현재 유효한 전체 메모리를 구성하여 reader에 제공한다.

4.1 Baselines

Recursive Summary Recursive Summary는 재귀 요약 접근을 개별 발화 턴에 적용한 기준선으로, 이전 전체 메모리와 현재 발화를 바탕으로 UPDATE 시 갱신된 전체 메모리를 재생성한다 (Wang et al., 2025). 변경되지 않은 내용도 재생

성하므로 메모리 크기에 따라 출력 비용이 증가한다.

Mem0 Mem0는 현재 발화에서 기억할 사실을 추출하고, 기존 메모리와 비교하여 추가·갱신·삭제 여부를 결정하는 방식이다 (Chhikara et al., 2025).

Patch Patch는 이전 전체 메모리와 현재 발화를 입력받아 ADD, REPLACE, DELETE 연산만 생성하며, 규칙 기반 실행기가 이를 즉시 적용한다. 다음 턴에는 수정된 전체 메모리를 입력하므로, 수정 위치에 따라 prefix KV-cache의 재사용 범위가 달라진다.

4.2 CAMP

CAMP는 메모리를 base memory (B_t)와 순서가 보존된 미병합 변경 연산인 pending updates (P_t)로 나누어 유지한다. 현재 유효한 메모리 (M_t)는 다음과 같이 정의한다.

$$M_t = \text{Apply}(B_t, P_t),$$

여기서 Apply는 pending 연산을 base에 순서대로 적용하는 결정론적 실행이다. 메모리 갱신 모델은 base, pending, 현재 발화를 순서대로 입력받아 변경 연산을 생성하고 이를 pending에 누적한다. 기억 활용 시에는 (M_t)를 reader에 제공하므로, 변경사항은 병합 전에도 즉시 활용된다. 모델이 예측한 UPDATE가 (k)회 누적되면 추가적인 모델 호출 없이 (M_t)를 새 base로 설정하고 pending을 비운다. 이 병합은 유효한 메모리 내용을 유지하면서 입력 표현을 변경한다.

CAMP는 병합 전까지 base를 안정적인 prompt prefix로 유지하여 턴 간 KV-cache 재사용을 높일 수 있다. 반면 pending의 누적은 입력 길이와 해석 부담을 늘리고, 병합 후에는 base 변경에 따른 입력 재계산이 필요하다. 병합 주기 (k)는 이러한 비용과 캐시 재사용 이점 사이의 균형을 조절한다.

5 Experimental Evaluation

5.1 Experimental Setup

실험은 메모리 갱신 방식이 기억의 정확성과 처리 비용에 미치는 영향을 평가하도록 구성하였다. CAMPBench 데이터에서 주요 성능을 비교하고, 인간 작성 평가 세트인 HST·HCT에서 추가 학습 없이 일반화 성능을 확인하였다. 모델 계열과 크기에 따른 차이를 살펴보기 위해 On-device에서 돌리기에 적합한 Granite 350M·1B, Qwen 0.8B·2B, Llama 1B·3B의 여섯 모델을 평가하였다 (IBM Granite Team, 2025; Qwen Team, 2026; Meta AI, 2024).

비교 기준선은 대화 정보의 보존과 갱신 방식에 따라 구성하였다. No Memory와 Recent-window는 각각 기억 없이 응답하는 경우와 최근 768 tokens만 활용하는 경우를 나타낸다. KV/LWW는 정보를 key-value 형태로 저장하고 동일한 key의 값을 최신 정보로 대체한다. 여기에 Section 4의 Mem0, Recursive Summary, Patch, CAMP를 포함하여 팩트 추출, 전체 재작성, 즉시 부분 수정, 변경분 누적 방식의 차이를 평가한다.

No Memory, Recent-window, KV/LWW는 Quiz를 푸는 톨 활용 방식만 학습하였고, Mem0, Recursive Summary, Patch, CAMP는 Quiz를 푸는 톨 활용 방식과 동일한 턴별 정답 궤적을 각 방법의 입력·출력 형식으로 변환하여 학습하였다. 모델 미세조정에는 LoRA를 사용하였다. (Hu et al., 2022)

기억의 품질은 갱신 판단, 상태 유지, 실제 활용의 세 측면에서 측정한다. Update F1은 UPDATE를 양성으로 하여 필요한 갱신을 놓치거나 불필요한 갱신을 수행하는 오류를 평가한다. Final-memory F1은 대화 종료 시 유지된 정보가 정답 메모리와 얼마나 일치하는지 측정한다. Quiz ESM은 예측 도구 호출과 정답의 정확 일치율로, 저장된 기억이 실제 요청 수행으로 이어지는지를 평가한다. 세 측면을 함께 비교하기 위해 다음의 가중 종합 점수를 사용한다.

$$S_{\text{comp}} = 0.60 \text{ ESM}_{\text{quiz}} + 0.25 F_{1,\text{final}} + 0.15 F_{1,\text{update}}.$$

Composite는 기억 활용 성능에 가장 큰 비중을 두면서 상태 유지와 갱신 판단을 함께 반영한다. 각 지표의 차이가 종합 점수에 가려지지 않도록 개별 지표도 함께 분석한다. No Memory와 Recent-window는 지속 메모리를 생성하지 않으므로 Quiz ESM만 보고한다.

처리 비용은 변경사항을 생성하는 시간뿐 아니라, 변경된 메모리로 인해 다음 턴에서 발생하는 입력 재계산 비용까지 고려한다. 이를 합산한 UPDATE-cycle latency를 사용하여, 갱신 방식에 따른 출력 비용과 후속 입력 처리 비용의 차이를 비교한다.

5.2 Overall Memory Performance

5.2.1 Overall performance

Table 1은 CAMPBench 테스트 데이터에서의 전체 비교 결과이다. Patch는 여섯 모델 모두에서 가장 높은 Composite를 기록하였으며, 평균 Quiz ESM에서도 가장 높은 성능을 보였다. Mem0는 여섯 모델 중 네 모델에서 가장 높은 Quiz ESM을 기록하여 기억 활용 측면에서 강점을 보였다. 평균 Composite는 Patch, CAMP, Summary, Mem0

Reader Model	No Memory	Recent-window	KV/LWW	Mem0	Summary	Patch	CAMP
Granite4 (350M)	-19.17	-51.67	42.71/54.06	58.56/ 67.40	55.28/55.62	59.62 /60.73	59.01/60.21
Qwen3.5 (0.8B)	-19.79	-58.75	42.65/53.96	65.50/ 77.29	64.38/67.29	70.39 /74.17	66.68/69.69
Granite4 (1B)	-26.35	-63.44	45.84/59.27	54.31/61.98	69.90/76.25	71.74 / 77.92	67.38/70.73
LLama-3.2 (1B)	-20.83	-54.90	41.21/51.56	62.74/ 73.33	60.30/60.83	67.06 /71.98	65.91/67.29
Qwen3.5 (2B)	-22.71	-63.44	46.40/60.21	61.89/71.98	65.61/69.69	71.13 / 75.83	69.10/71.67
LLama-3.2 (3B)	-24.90	-62.40	45.21/58.23	64.00/ 74.90	66.55/69.79	71.06 /74.27	69.22/72.71
Mean	-22.29	-59.10	44.00/56.22	61.17/71.15	63.67/66.58	68.50 / 72.48	66.22/68.72

Table 1: Performance on the synthetic evaluation data. Each cell reports Composite / Quiz ESM. The mean is computed across the six reader models. G, Q, and L denote Granite, Qwen, and Llama, respectively. Bold indicates the best result for each metric.

Method	Immediate	Delayed	Composite	Final
Summary	74.62	61.02	45.34	67.01
Mem0	78.30	64.38	52.69	73.12
Patch	79.74	66.09	53.41	74.17
CAMP	75.47	62.30	50.36	70.97

Table 2: Average Quiz ESM (%) by Quiz Placement

Evaluation Set	Summary	Mem0	Patch	CAMP
HST	70.37	57.29	73.07	76.93
HCT	32.75	31.69	34.41	33.80

Table 3: Average Quiz ESM (%) on Human-Written Evaluation Data

순으로 높았으며, 네 방법 모두 평균 Quiz ESM에서 No Memory, Recent-window, KV/LWW를 상회하였다.

지표에 따른 방법 간 순위 차이는 기억 활용 성능과 메모리 관리 성능을 구분하여 살펴볼 필요가 있음을 보여준다. Mem0는 평균 Quiz ESM에서 Patch에 이어 두 번째로 높았지만, 갱신 판단과 최종 메모리 상태의 정확성을 함께 반영한 Composite에서는 CAMP와 Summary보다 낮았다. 이러한 정확도 비교만으로는 각 방법의 온디바이스 적용 시 절충 관계를 판단하기 어렵다. 이에 Section 5.2, 5.3에서는 전반적으로 높은 성능을 보인 네 방법을 대상으로, 메모리 갱신과 후속 입력 처리의 지연시간을 함께 비교한다.

5.2.2 Performance by quiz placement

CAMPBench의 턴별 평가를 활용하여 Quiz 성능을 Immediate Turn, Delayed Turn, Composite Turn 및 Final Quiz로 구분하였다. 이를 통해 최종 시점의 성능뿐 아니라, 대화가 진행되는 과정에서 저장된 신호를 언제, 어떤 조건에서 활용하는지 평가하였다. Table 2에서 Patch는 모든 Quiz 유형에서 가장 높은 성능을 기록하여, 전체 결과에서의 우위가 특정 평가 시점에 한정되지 않음을 보여주었다. 네 방법 모두 Immediate보다 Delayed에서 낮은 성능을 보였으며, 여러 신호를 함께 적용하는 Composite Turn에서 가장 큰 어려움을 보였다. 이러한 결과는 CAMPBench의 턴별 평가가

최종 성능만으로는 드러나지 않는 중간 시점의 성능 저하와 복합 신호 처리의 어려움을 구분하여 보여준다는 점을 확인한다.

Quiz 유형마다 요청 구성이 다르므로 Delayed의 성능 저하만으로 정보 소실을 단정하지 않으며, 구체적인 오류 발생 과정은 Section 6.3의 턴별 trace 분석에서 살펴본다.

5.2.3 Human-written evaluation

평균 Composite가 높았던 네 방법인 Summary, Mem0, Patch, CAMP를 대상으로 후속 일반화 평가를 수행하였다. HST · HCT에서는 추가 학습 없이 이들 방법의 상대적 성능이 인간 작성 대화에서도 유지되는지 확인하였다. 인간 작성 평가 데이터에서의 결과는 Table 3에서 HST에서는 CAMP가 가장 높은 평균 Quiz ESM을 기록하여, CAMPBench 평가에서 나타난 Patch의 우위가 인간 작성 데이터에서 그대로 유지되지는 않았다. 다만 모델별 최우수 방법은 CAMP 3개, Patch 2개, Summary 1개로 나뉘어 방법 간 우위가 모델에 따라 달랐다. HCT에서는 Patch의 평균이 가장 높았지만 CAMP와의 차이는 0.61%p에 불과했다. 또한 네 방법 모두 HST보다 낮은 성능을 보여, 사용자 간 신호 충돌과 복합 요청을 포함하는 환경에는 여전히 개선의 여지가 있음을 확인하였다.

Model	Summary	Mem0	Patch	CAMP
Granite 350M	55.28 / 3.95 [100.0]	58.56 / 4.77 [120.8]	59.62 / 2.62 [66.4]	59.01 / 2.17 [54.9]
Qwen 0.8B	64.38 / 11.87 [100.0]	65.50 / 11.87 [100.0]	70.39 / 8.83 [74.4]	66.68 / 6.03 [50.8]
Granite 1B	69.90 / 20.49 [100.0]	54.31 / 21.34 [104.2]	71.74 / 12.75 [62.2]	67.38 / 9.67 [47.2]
Llama 1B	60.30 / 12.48 [100.0]	62.74 / 17.42 [139.6]	67.06 / 9.29 [74.4]	65.91 / 6.92 [55.5]
Qwen 2B	65.61 / 25.39 [100.0]	61.89 / 28.29 [111.4]	71.13 / 21.39 [84.2]	69.10 / 15.95 [62.8]
Llama 3B	66.55 / 32.68 [100.0]	64.00 / 51.88 [158.8]	71.06 / 27.55 [84.3]	69.22 / 20.91 [64.0]

Table 4: Composite score and UPDATE-cycle latency by memory update method. Each cell reports Composite / latency (s) [relative latency (%)]. Relative latency is normalized to Summary (= 100) for the same model. Bold indicates the highest Composite score and the lowest latency for each model.

5.3 On-Device Efficiency and Accuracy-Latency Trade-offs

다음으로, 주요 네 가지 방법의 온디바이스 메모리 처리 비용을 평가하였다. 메모리 관리가 대화 중 반복적으로 수행되는 환경에서는 정확도뿐 아니라 갱신과 후속 입력 처리에 필요한 비용도 중요하다. 이를 위해 UPDATE를 적용한 턴의 생성 비용과 직후 턴의 입력 재계산 비용을 합산한 UPDATE-cycle latency를 측정하고 Composite와 함께 비교하였다. CAMP는 기본 병합 주기 $k = 5$ 를 사용하였다. 가장 일관된 차이는 UPDATE가 발생했을 때 전체 메모리를 다시 생성하는 방식과 변경 연산만 생성하는 방식 사이에서 나타났다. Table 4에서 Patch와 CAMP는 Summary 대비 UPDATE-cycle latency를 각각 약 16–38%와 36–53% 줄였다. 특히 Patch는 모든 모델에서 Summary보다 높은 Composite와 낮은 latency를 함께 기록하여, 변경 부분만 수정하는 방식이 정확도와 처리 비용 모두에서 유리한 결과를 보였다.

Mem0의 UPDATE-cycle latency는 Summary의 1.00–1.59배로, Summary 대비 일관된 속도 이점을 보이지 않았다. CAMP는 모든 모델에서 Patch보다 낮은 UPDATE-cycle latency를 기록했으며, 절감률은 모델별 17–32%, 여섯 모델 평균 24.7%였다. Patch와 CAMP가 모두 변경 연산만 생성하는 가운데, CAMP는 기존 메모리를 안정적인 prefix로 유지함으로써 직후 입력에서 재계산해야 하는 범위를 추가로 줄였다.

이러한 속도 이점에는 정확도 측면의 절충이 동반되었다. CAMP의 Composite는 Patch보다 모델별로 0.61–4.36점 낮았다. 따라서 동일 모델에서 종합 정확도를 우선하면 Patch가, 갱신 latency를 우선하면 CAMP가 유리했다. 다만 앞선 결과에서 CAMP가 일부 모델의 Final-memory F1과 HST 성능에서 Patch를 상회했으므로, Composite 차이를 모든 기억 능력의 일관된 저하로 해석하

기는 어렵다.

종합하면 Patch와 CAMP는 Summary 및 Mem0보다 높은 평균 Composite와 낮은 UPDATE-cycle latency를 보여, 정확도와 처리 비용을 함께 고려할 때 경쟁력 있는 결과를 보였다.

6 Ablation and Analysis

이러한 결과를 바탕으로, Patch와 CAMP의 효율성 차이와 정확도-지연시간 절충, 오류 양상을 상세히 분석한다.

6.1 Effect of Prefix-Cache Reuse

CAMP의 효율성 이점이 안정적인 prefix의 재사용에서 발생하는지 확인하기 위해, Base, U40, U60, U80 workload에서 Patch와 CAMP를 Cache ON/OFF 조건으로 비교하였다. 실험은 동일한 24개 테스트 시나리오와 정답 상태 전이를 사용하여 자동차에 들어갈 수 있는 실제 모바일 기기 (Samsung Galaxy Tab S11)에서 수행하였다. CAMP는 기본 병합 주기 $k=5$ 를 사용하였으며, UPDATE-cycle latency는 UPDATE 생성과 직후 턴의 입력 처리 비용을 포함한다.

Figure 2(a)는 각 모델과 workload의 Patch Cache-OFF latency를 100으로 정규화한 결과이다. Cache OFF에서 CAMP는 Patch보다 1.9–4.3% 느려 별도의 속도 이점을 보이지 않았다. 반면 Cache ON에서는 CAMP가 Patch보다 Base에서 24.7%, U40에서 44.5%, U60에서 52.7%, U80에서 57.1% 낮은 latency를 기록하였다. 즉, Patch와 CAMP의 차이는 cache를 사용할 때 나타났으며, UPDATE 부하가 증가할수록 확대되었다.

Figure 2(b)는 이러한 차이를 prefix 재사용률로 분석한다. Patch의 post-UPDATE prefix 재사용률은 Base의 41.8%에서 U80의 25.4%로 감소한 반면, CAMP는 같은 구간에서 60.9%에서 73.1%로 증가하였다. Patch는 유효 메모리를 직접 수정하므로, 최초 변경 토큰의 위치에 따라 후속 요청에서 재사용할 수 있는 공통 prefix가 제한된다.

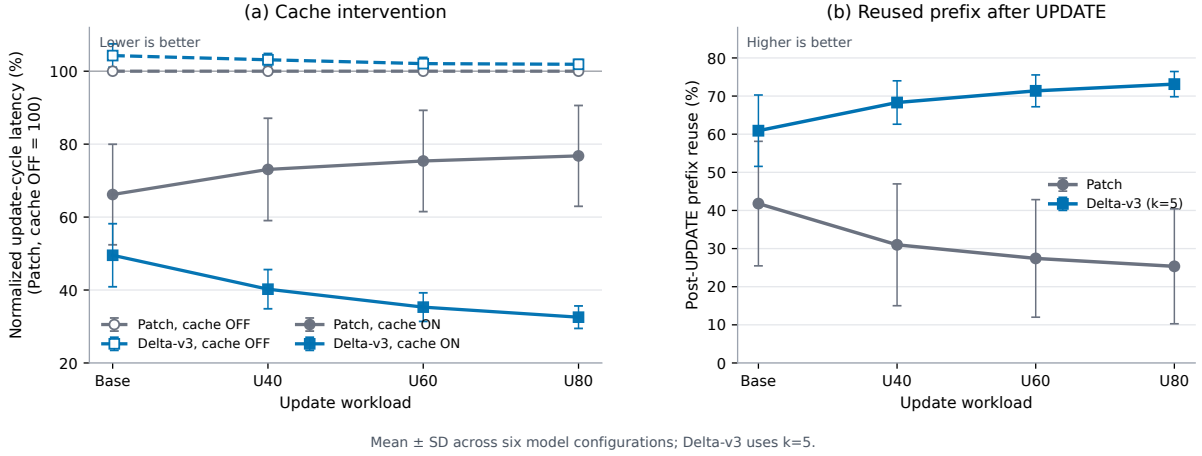


Figure 2: Prefix-cache 재사용 효과. (a) Patch Cache OFF를 100으로 정규화한 UPDATE-cycle latency. (b) UPDATE 직후 재사용된 prefix의 비율. 점과 오차막대는 각각 여섯 모델의 평균과 표준편차를 나타내며, CAMP는 $k = 5$ 를 사용한다.

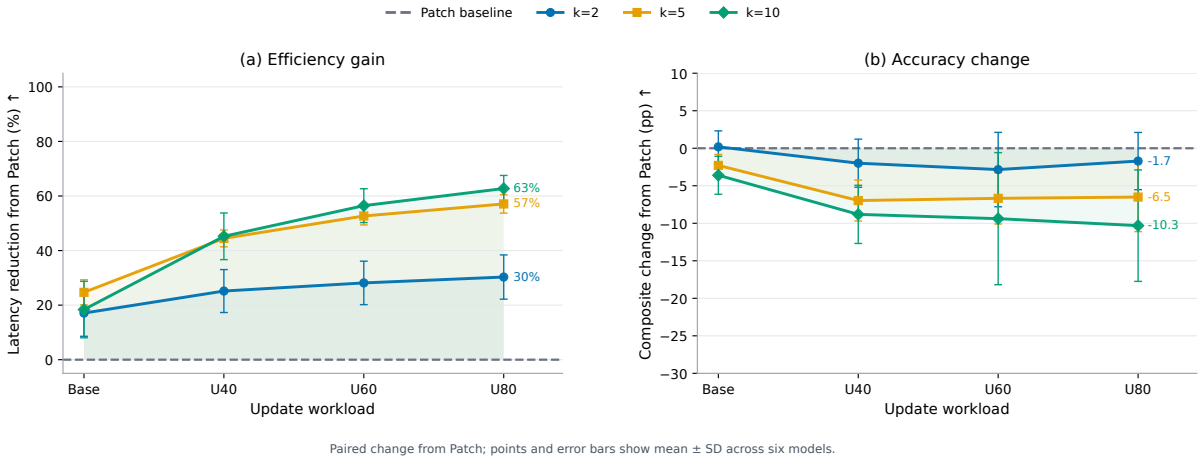


Figure 3: Effect of the compaction interval. (a) Cache-ON latency relative to Patch and (b) Composite under increasing update load. Lines and shaded regions show the mean and standard deviation across six models. Larger k reduces latency more aggressively but incurs greater accuracy loss.

CAMP는 변경분을 base memory 뒤에 누적하여 병합 전까지 기존 prefix를 유지하므로, UPDATE가 반복되어도 입력의 상당 부분을 재사용할 수 있다.

CAMP의 logical prompt는 Patch보다 3.5–11.7% 길었지만, 실제로 재계산한 post-UPDATE prefill은 24.4–62.3% 적었다. 따라서 관찰된 latency 감소는 짧은 입력 자체보다 안정적인 prefix의 KV-cache 재사용에서 주로 발생한 것으로 볼 수 있다. 이러한 결과는 CAMP가 특히 UPDATE가 빈번한 환경에서 효율적인 운영점을 제공한다는 점을 보여준다.

Cache 유지에 따른 부가 비용도 함께 측정하였다. 병합과 base prefix 재구축 비용을 포함한 조건에서, 대표적인 Granite 4 350M 구성의 cache 관리 시간은 전체 native latency의 0.02% 미만이었다.

있으며 Cache ON에서도 peak process memory가 일관되게 증가하지 않았다. 이런 경향은 타 모델에서도 일관된 패턴을 보였다. 따라서 prefix-cache 재사용에 따른 latency 절감이 별도의 큰 관리 및 메모리 비용으로 상쇄되지는 않았다. 세부 workload별 PSS/RSS와 병합 비용은 Appendix에서 보고한다.

6.2 Effect of Compaction Interval

Section 6.1에서는 CAMP의 효율성 이점이 안정적인 prefix의 cache 재사용에서 발생함을 확인하였다. 여기서는 cache를 활성화한 상태에서 병합 주기 k 를 변경하여, prefix를 유지하는 기간이 정확도와 지연시간에 미치는 영향을 분석한다. Patch와 $k \in \{2, 5, 10\}$ 을 Base부터 U80까지 비교하였으며, Figure 3 (a)와 (b)는 각각 Patch 대비

Metric	Patch	CAMP
Immediate new-fact acquisition	34.07%	34.74%
Final retention of acquired facts	96.43%	97.02%
First loss on later NO_OP	0.26%	0.00%
First loss on later UPDATE	3.30%	2.98%
False UPDATE on gold NO_OP	2.23%	1.21%

Table 5: Timing and Transition Patterns of Memory Errors

latency 절감률과 Composite 변화량을 보여준다.

병합을 늦출수록 CAMP의 latency 이점은 UPDATE 부하와 함께 커졌다. 반면 Figure 3 (b)에서 긴 병합 주기는 더 큰 정확도 감소를 동반하였다. $k = 2$ 는 전반적으로 Patch 정확도의 대부분을 유지하면서 일관된 latency 절감을 제공했고, $k = 5$ 와 $k = 10$ 은 정확도 비용을 감수하는 대신 고 부하에서 더 큰 효율성을 제공하였다. 특히 U80 평균에서 네 설정은 서로 다른 비지배 operating point를 형성하여, 어느 하나가 모든 정확도와 비용 기준에서 우월하지 않았다.

이 결과는 CAMP의 병합 주기를 통해 시스템의 정확도-지연시간 예산을 조절할 수 있음을 보여준다. $k = 2$ 는 Patch의 정확도를 대부분 유지하면서 일정한 latency 절감을 제공하는 conservative operating point이다. 반면 $k = 5$ 는 더 큰 latency 절감을 제공하면서도 $k = 10$ 에서 나타나는 추가적인 정확도 저하를 피하여, 본 연구의 기본 설정으로 적절한 균형을 보였다. $k = 10$ 은 고부하 조건에서 가장 낮은 평균 latency를 기록했지만, $k = 5$ 대비 추가 절감 폭은 상대적으로 제한적이고 모델에 따라 일관되지 않은 반면 Composite 감소는 더 크게 나타났다. 따라서 $k = 10$ 은 latency 제약이 매우 강한 경우에 적합한 공격적 설정으로 해석할 수 있다.

6.3 Error Dynamics and Failure Analysis

6.3.1 Error Dynamics: When Memory Errors Arise

메모리 오류의 발생 시점을 확인하기 위해 Patch와 CAMP의 턴별 trace를 분석하였다. Table 5에서 두 방법의 신규 fact 즉시 획득률은 사용자 · 시점 · 구조화된 값의 완전 일치율 기준으로 약 34%였으며, 정확히 획득된 fact의 96–97%는 교체 · 삭제 또는 시나리오 종료까지 유지되었다. 다만 이 유지율에는 모델이 NO_OP를 예측하면 기존 메모리를 그대로 보존하는 설계도 반영된다. 획득 이후의 정보 손상은 주로 후속 UPDATE와 함께 관찰되었다. 이러한 결과는 본 설정에서 최초 정

보 획득과 후속 갱신의 정확성을 개선할 필요가 있음을 시사한다.

6.3.2 Failure Analysis: 갱신 오류 사례

동일한 Quiz에서 한 방법만 오답을 생성한 사례를 대상으로, 메모리 상태와 정답 근거, 갱신 기록을 비교하였다. 여섯 모델의 사례를 합산한 CAMP 661건과 Patch 442건의 유효 단독 실패 집합에서, 정보 누락과 오래된 값의 유지가 각각 약 82%를 차지하였다. 각 집합 내 충돌 유형의 비중은 CAMP에서 8.9%, Patch에서 5.4%였다. 상세 오류 분포와 추가 사례는 부록에 제시한다.

CAMP의 삭제 대상 지정 오류는 갱신 실패가 기억 활용에 미치는 영향을 보여준다. 폭풍 종료 후 Helena의 평상시 온도인 22도로 복원하는 사례에서, CAMP는 base에 남은 임시 설정 24도를 삭제해야 했으나 무관한 door-lock 정보를 삭제 대상으로 생성하였다. 그 결과 22도와 24도가 함께 남았고 reader는 24도를 선택한 반면, Patch는 22도를 유지하여 정답을 생성하였다. 이 사례는 base와 pending으로 표현된 메모리에서 현재 유효한 상태를 파악하고 삭제 대상을 정확히 지정하는 과정의 중요성을 보여준다.

7 Conclusion

본 연구는 온디바이스 개인화 비서의 메모리 관리를 평가하기 위한 CAMPBench와, 안정적인 prefix 및 변경분의 주기적 병합을 활용하는 CAMP를 제안하였다. 여섯 소형 언어모델의 성능 평가와 모바일 기기 실험을 통해 prefix KV-cache 재사용이 메모리 갱신 지연시간을 줄이며, 병합 주기로 정확도-지연시간 절충을 조절할 수 있음을 확인하였다. 또한 턴별 오류 분석에서는 정확히 획득한 정보의 장기 보존보다 최초 갱신 시 필요한 정보를 정확하게 기록하는 과정이 주요 병목으로 나타났다. 향후에는 정보 획득의 정확성을 높이고, 갱신 부하와 메모리 상태에 따라 병합 시점을 적응적으로 결정하는 방향으로 확장할 수 있다.

References

- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. [Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve](#). In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, Santa Clara, CA. USENIX Association.
- Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S. Khatamifard, Minsik Cho, Carlo C. Del

673	Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient large language model inference with limited memory . In <i>Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 12562–12584, Bangkok, Thailand. Association for Computational Linguistics.	730
674		731
675		732
676		733
677		734
678		735
679		736
680	Apple. 2024. Apple intelligence foundation language models . <i>Preprint</i> , arXiv:2407.21075.	737
681		738
682	Yuhao Chen, Yi Xu, Xinyun Ding, Xiang Fang, Shuochen Liu, Luxi Lin, Qingyu Zhang, Ya Li, Quan Liu, and Tong Xu. 2026. VehicleMemBench: An executable benchmark for multi-user long-term memory in in-vehicle agents . <i>Preprint</i> , arXiv:2603.23840.	739
683		740
684		
685		
686		
687	Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory . In <i>ECAI 2025</i> , Frontiers in Artificial Intelligence and Applications. IOS Press.	741
688		742
689		743
690		744
691		
692	Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. LightMem: Lightweight and efficient memory-augmented generation . In <i>International Conference on Learning Representations</i> , pages 98706–98729.	745
693		746
694		747
695		748
696		749
697		750
698		751
699	In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference . In <i>Proceedings of Machine Learning and Systems</i> , volume 6, pages 325–338.	752
700		753
701		754
702		755
703		756
704	Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models . In <i>International Conference on Learning Representations</i> .	757
705		758
706		759
707		760
708		761
709	IBM Granite Team. 2025. Granite 4.0 Nano Language Models . Official Hugging Face model collection. Models used: ibm-granite/granite-4.0-350m and ibm-granite/granite-4.0-1b; accessed 2026-09-21.	762
710		763
711		764
712		765
713	Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of LLM agents . In <i>Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics.	766
714		767
715		768
716		769
717		770
718		771
719		772
720		773
721	Meta AI. 2024. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models . Official model release. Models used: meta-llama/Llama-3.2-1B-Instruct and meta-llama/Llama-3.2-3B-Instruct; accessed 2026-09-21.	774
722		775
723		776
724		777
725		778
726	Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems . <i>Preprint</i> , arXiv:2310.08560.	779
727		780
728		781
729		782
	Ramit Pahwa, Apoorva Beedu, Parivesh Priye, Rutu Gandhi, Saloni Takawale, Aruna Baijal, and Zengli Yang. 2026. Audio2Tool: Speak, call, act—a dataset for benchmarking speech tool use . <i>Preprint</i> , arXiv:2604.22821. Accepted at Interspeech 2026; official proceedings entry was not yet available when this bibliography was verified.	
	Qwen Team. 2026. Qwen3.5: Towards native multi-modal agents . Official model release. Models used: Qwen/Qwen3.5-0.8B and Qwen/Qwen3.5-2B; accessed 2026-09-21.	
	Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A temporal knowledge graph architecture for agent memory . <i>Preprint</i> , arXiv:2501.13956.	
	Sarah Theres V"olkel, Daniel Buschek, Malin Eiband, Benjamin R. Cowan, and Heinrich Hussmann. 2021. Eliciting and analysing users' envisioned dialogues with perfect voice assistants . In <i>Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems</i> , pages 1–15, New York, NY, USA. Association for Computing Machinery.	
	Qingyue Wang, Yanhe Fu, Yanan Cao, Shuai Wang, Zhiliang Tian, and Liang Ding. 2025. Recursively summarizing enables long-term dialogue memory in large language models . <i>Neurocomputing</i> , 639:130193.	
	Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking chat assistants on long-term interactive memory . In <i>International Conference on Learning Representations</i> , pages 86809–86836.	
	Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic memory for LLM agents . In <i>Advances in Neural Information Processing Systems</i> , volume 38, pages 17577–17604. Curran Associates, Inc.	
	Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient execution of structured language model programs . In <i>Advances in Neural Information Processing Systems</i> , volume 37, pages 62557–62583. Curran Associates, Inc.	

8 Appendix

8.1 데이터 구축 방식 검증

CAMPBench에 사용한 Hybrid 방식의 선택 근거를 확인하기 위해, 동일한 persona와 구조화된 사건을 고정된 20개 시나리오에서 세 가지 데이터 구축 방식을 비교하였다. Post-hoc은 전체 대화를 생성한 뒤 메모리 갱신 시점을 사후 정렬하며, Native turn-wise는 매 턴의 대화 생성과 메모리

Method	Update Audit PASS (%)	Answer support (%)	ESM (%)	Cost (\$/scenario)
Post-hoc	94.1	94.0	89.2	2.58
Native turn-wise	68.6	95.5	90.2	21.55
Hybrid	95.3	96.5	91.7	2.55

Table 6: Comparison of quality and generation cost across data construction methods

Failure Cause	CAMP Failure	Patch Failure
MISSING_FACT	420 (63.5%)	290 (65.6%)
STALE_VALUE	119 (18.0%)	71 (16.1%)
CONFLICTING_FACTS	59 (8.9%)	24 (5.4%)
WRONG_OWNER	35 (5.3%)	22 (5.0%)
CONDITION_LOSS	12 (1.8%)	9 (2.0%)
EVIDENCE_IGNORED	16 (2.4%)	26 (5.9%)
Total	661 (100%)	442 (100%)

Table 7: Distribution of causes in method-specific failures. Percentages use each method’s exclusive-failure set as the denominator.

상태 확정을 순차적으로 수행한다. Hybrid는 사건 단위로 대화를 일괄 생성하되, 다음 사건으로 넘어가기 전에 내부 발화를 턴별로 처리하여 메모리 상태를 확정한다.

평가는 턴별 갱신 라벨의 감사 통과율(Update Audit PASS), Quiz 정답의 메모리 근거 충족률(Answer support), 고정된 reader의 도구 실행 정확도(ESM), 시나리오당 생성 비용으로 수행하였다. 비교 결과는 표 6에 제시한다. Persona와 구조화된 사건은 동일하지만, 각 방식이 생성하는 대화와 Quiz는 서로 다르다. 따라서 ESM 차이는 생성된 데이터의 구성과 난이도도 영향을 줄 수 있으며, 이를 데이터 구축 결과의 활용 가능성을 살펴보는 보조 지표로 해석한다.

Hybrid는 본 비교에서 세 품질 지표 모두 가장 높은 관측값을 보였으며, 시나리오당 생성 비용은 \$2.55로 Post-hoc의 \$2.58과 유사하였다. Native turn-wise는 생성 비용이 더 높았으며, 해당 구현에서는 근거가 확정되기 전의 초기 갱신과 갱신 누락이 관찰되었다.

이 비교에서 확인한 갱신 라벨의 감사 통과율, 정답 근거 충족률 및 생성 비용을 고려하여 CAMPBench 구축에 Hybrid를 채택하였다.

8.2 Failure taxonomy: Quiz 시점에서 오류가 어떤 상태로 관찰됐는가

본문의 오류 발생 시점 분석을 보완하여, 여기서는 Quiz 시점에서 관찰된 오류 유형과 구체적인

사례를 제시한다. 동일한 평가 사례에서 한 방법만 오답을 생성한 경우를 대상으로, Quiz 시점의 메모리와 정답 근거, 갱신 기록을 비교하여 주된 실패 원인을 분류하였다. 비율은 여섯 모델의 사례를 합산한 뒤, 각 방법의 유효 단독 실패 집합(CAMP 661건, Patch 442건)을 분모로 계산하였다. 두 단독 실패 집합 모두에서 정보 누락이 가장 빈번했으며, 오래된 값의 유지까지 포함하면 약 82%를 차지하였다. 이는 두 방법의 단독 실패 사례에서 정보의 획득 및 갱신과 관련된 오류가 주요 비중을 차지함을 보여준다. 턴별 trace 분석에서는 한번 정확히 획득된 정보의 단순한 NO_OP 구간 소실은 드물었다. 한편 충돌 유형은 CAMP 실패 측에서, 충분한 메모리 근거를 활용하지 못한 유형은 Patch 실패 측에서 상대적으로 높은 비중을 보였다. 다음 세 사례는 이러한 실패 양상을 구체적으로 보여준다.

공통 병목: 필요한 선호의 누락. “평상시 출퇴근 상황으로 돌아왔으니, 교통정보 표시를 평소 설정으로 복원해 줘”라는 요청에서, Patch는 평소 설정인 `traffic_display=false`를 보존하여 정답을 생성하였다. 반면 CAMP는 해당 선호를 누락하여 ‘true’를 출력하였다. 이는 두 단독 실패 집합에서 가장 빈번했던 정보 누락 유형의 CAMP 측 사례로, 필요한 사실이 최초 갱신에서 기록되지 않거나 후속 갱신 과정에서 누락되면, reader가 정답을 복원할 근거 자체가 부족해짐을 보여준다.

CAMP의 충돌: 삭제 대상 지정 오류. “폭풍이 끝났으니 Helena의 평상시 passenger 온도로 복원해줘”라는 요청의 정답은 22도였다. 앞선 폭풍 종료 턴에서 CAMP는 base에 남아 있는 임시 설정 24도를 삭제해야 했으나, 무관한 door-lock 정보를 삭제 대상으로 생성하였다. 그 결과 24도가 평상시 값 22도와 함께 남았고, reader는 24도를 선택하였다. Patch는 22도를 유지하여 정답을 생성하였다. 이 사례는 base와 pending 연산으로 표현된 메모리에서 현재 유효한 정보를 파악하고, 제거할 대상을 정확히 지정하는 과정의 중요성을 보여준다.

Patch의 근거 활용 실패: 올바른 메모리에서 잘못된 값 선택. 중앙 디스플레이 밝기 평가 사례에서는 정답과 Patch 메모리 모두 `brightness=50`을 명확히 포함했고, 관련 후보도 50 하나뿐이었다. 그러나 Patch의 reader는 과거 값인 60을 출력하였다. CAMP 역시 정답 정보 50을 보유하고 있었으며, 이를 올바르게 출력하였다. 이 사례는 메모리에 필요한 정보가 정확하게 남아 있어도 도구 호출 단계에서 활용되지 못할 수 있음을 보여준다.